Hernández Aviña Karen Itzel.

** PREGUNTAS ABIERTAS **

- Pregunta 0.1

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código ¿Qué es la abstracción al modelar un objeto? Da un ejemplo de un dato que conservarías y otro que omitirías según el contexto.

La abstracción es una manera de saber que cosas o características de un objeto son necesarias para modelarlas en un código en base a algún contexto, en el caso de un objeto como carro, podemos conservar cuál es la velocidad máxima, pero tal vez podemos descartar de que color es, ya que no es esencial para nuestro código.

- Pregunta 0.2

¿Para qué sirve el constructor y en qué se distingue de un método común en su declaración?

El constructor nos sirve para inicializar el estado de instancia, lleva el mismo nombre que la clase y permite que si creamos un objeto nuevo se adapte a la estructura que el tiene, se distingue de un método ya que no lleva ningún tipo de retorno, ni void.

- Pregunta 0.3

¿Qué impide final en una variable? Si guarda una referencia a un arreglo, ¿impide cambiar sus elementos?

final impide la reasignación, es decir, esa variable ya no puede apuntar a otro objeto o valor después de ser inicializada, como solamente protege la referencia (dirección de memoria) si podemos cambiar la información de los elementos, podemos modificar o eliminar los elementos del arreglo.

• Pregunta 0.4

¿Por qué == no basta para comparar el contenido de dos Strings? Explica qué usarías para comparar sus textos?

No basta ya que no compara el contenido de los String, compara las direcciones de memoria, para poder comparar el contenido debemos usar equals()

• Pregunta 0.5

¿En qué se distinguen while y do-while? Explica qué ocurre si la condición es falsa desde el inicio.

La diferencia es que en while primero evalua si la condición se cumple para entrar en esa parte del código, si es falsa no entra, en el do-while se ejecuta aunque sea una vez esa parte del código, y al final se evalua la condición, si es falsa, solamente se ejecuta una vez.

** R1 ¿Qué imprime?

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
int[][] tabla = {{4, 4, 7}, {2, 6, 8}};
int[] fila = tabla[0];
int total = 0;
for (int i = 0; i < fila.length; i++) {
    fila[i] = fila[i] / 2;
    total += tabla[0][i];
    System.out.println(i + ": " + total);
}
System.out.println((double)(total/2));
System.out.println("P" + fila[0] + tabla[1][0]);
```

fila = [9,4,7] i < 3

for (int i=0, i< fila.length ; i++){

i	fila[i]	Fila	tabla =	System
0	9/2 = 4 (entero)	[4,4,7]	[0][0] = 0+4 = 4	0:4
1	4/2 = 2	[4,2,7]	[0][1] = 4+2 = 6	1:6
2	7/2 = 3	[4,2,3]	[0][2] = 6+3 = 9	2:9

Fuera del for.

System.out.println((double)(tabla1/2)); → 1.0

(convierte a double) (9/2 = 4)

System.out.println("P"+fila[0]+tabla[1][0]); → P42

(fila[0] = 4, tabla[1][0] = 2)

Salida final

0:4

1:6

2:9

4.0

P42

** R2 a). ¿Compila, truena o Funciona?.

```
int[] datos = {3,6};
int total=0;
for(int i=0; i<=datos.length; i++)
   total+=datos[i];
System.out.println(total);
```

Respuesta: Compila pero truena en ejecución, compila ya que la sintaxis es la correcta, pero truena por la condición en el for i<=datos.length, ya que al recorrer el arreglo hace datos[2], lo cual está fuera del arreglo.

** R2 b) ¿Compila, truena o Funciona?.

```
Integer piezas = 6;
System.out.println(piezas+2);
System.out.println("P"+piezas);
```

Respuesta: Funciona, ya que integer se puede usar como entero para sumar, en el primer println se suma 6+2 y en el segundo se concatena piezas con P, lo que nos da P6, imprime 8/P6

** R2 c) ¿Compila, truena o Funciona?.

```
static int medir(int valor){
   if (valor>0) return valor;
}
```

Respuesta: No compila, en Java si un método retorna algo necesariamente tenemos que hacer que retorne algo, aquí si valor es cero o negativo no hay algo que retornar, por lo que necesitamos algo que retornar si no se entra en el if.

★★★ B3. Modelado: base, reto y justificación

Cada lámpara tiene etiqueta fija y nace apagada. Cuenta las lámparas creadas. Escribe encender() y una consulta del estado; cada lámpara cambia por separado. Usa el enum dado.

enum Estado {APAGADO, ENCENDIDO, PARPADEO}

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta. ¿Qué tipo tiene el atributo de estado y con qué valor nace? Justifica una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos. Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad

Clase:

```
public class Lampara {

    final String etiqueta;
    Estado estado = Estado.APAGADO;
    static int creadas = 0;

    Lampara (String etiqueta) {
        this.etiqueta = etiqueta;
        this.estado = Estado.APAGADO;
        creadas++;
    }

    void encender() {
        this.estado = Estado.ENCENDIDO;
    }

    Estado consultar() {
        return this.estado;
    }
}
```

Justificaciones

* Tipo: String para la etiqueta y Estado por el enum que se nos da.
* Instancia/clase: etiqueta y estado de instancia (una para cada objeto), creadas es de clase.
* Final / mutable: etiqueta es final y estado y creadas mutables.
* Constructor: Recibe como parámetro la etiqueta, estado inicializado en apagado y se incrementa en 1 las lámparas creadas.
* Métodos: encender() no returna nada pero cambia el estado y consultar() nos returna el estado de la lampara.

*** R1 - Código con repetición ***

Devuelve la posición de la menor temperatura usando for; si se repite, devuelve la primera. Los índices empiezan en 0. El arreglo no es null, tiene al menos un elemento y puede incluir negativos. No lo modifiques.

static int posicionMinima(int[] temperaturas)

Ejemplos (argumentos → retorno):

{8, 3, 5} → 1 · {1} → 0 · {-2, -8, 1} → 1

Escribí el método completo. No imprimas ni leas del teclado.

```
static int posicionMinima(int[] temperaturas) {
int posMinima = 0; → (recordar posición, no valor)
for(int i = 1; i < temperaturas.length; i++) {
    if(temperaturas[i] < temperaturas[posMinima]) {
        posMinima = i;
    }
}
return posMinima;
}
```

Compara si el elemento que esta antes de posMinima (el valor en esa posición) es menor, si es menor toma la posición que sigue, el ciclo termina cuando i recorre todos los elementos del arreglo y por último nos retorna la posición.

Con [8, 3, 5]

int posMinima = 0

	i	for(...)	if(...)	posMinima
①	1	1<3 ✓	if(3<8) ✓	1
②	2	2<3 ✓	if(5<3) X	1
③	3	3<3 X	No entra	1

Fuera del for

return 1

{8, 3, 5}
0 1 2 ✓

***RS · código con selección

...n equipo desde su mantenimiento según los ciclos de uso, Un valor menor o igual que ... es "INVALIDO". Para los positivos, aplica la primera regla que corresponda, en el ...den de la tabla. Usa if/else-if/else. No imprimas ni trag del teclado.

static String mantenimiento (int ciclos) {

Regla o valor recibido	Texto devuelto
Menor o igual a 0	"INVALIDO"
Divisible entre 24	"COMPLETO"
Si no, divisible entre 12	"REVISION"
Si no, divisible entre 4	"BASICO"
Cualquier otro	"NINGUNO"

```
static String mantenimiento (int ciclos) {
  if (ciclos ≤ 0) {
    return "INVALIDO";
  } else if (ciclos % 24 == 0) {
    return "COMPLETO";
  } else if (ciclos % 12 == 0) {
    return "REVISION";
  } else if (ciclos % 4 == 0) {
    return "BASICO";
  } else {
    return "NINGUNO";
  }
}
```

ciclos	if (ciclos ≤ 0)	if (ciclos % 24 == 0)	if (ciclos % 12 == 0)	if (ciclos % 4 == 0)
-5	(-5 ≤ 0) ✓ return "INVALIDO"	—	—	—
24	(24 ≤ 0) ✗	(24 % 24 == 0) ✓ return "COMPLETO"	—	—
84	(84 ≤ 0) ✗	(84 % 24 == 0) ✗	(84 % 12 == 0) ✓ return "REVISION"	
8	(8 ≤ 0) ✗	(8 % 24 == 0) ✗	(8 % 12 == 0) ✗	(8 % 4 == 0) ✓ return "BASICO"
10	(10 ≤ 0) ✗	(10 % 24 == 0) ✗	(10 % 12 == 0) ✗	(10 % 4 == 0) ✗

return "NINGUNO"